

Mini Project Code:

```
# CAP776 - Minor Project 1
```

```
# My Data, My Story (Aman Sharma 12608986)
```

```
import openpyxl
```

```
from datetime import date
```

```
file_name = "1234567_updated.xlsx"
```

```
sheet_name = "Daily Log"
```

```
start_row = 6
```

```
# the recording window (first and last date in my Excel sheet)
```

```
record_start = date(2026, 7, 20)
```

```
record_end = date(2026, 9, 25)
```

```
# turning words into numbers so I can do maths with them (scale given by sir)
```

```
feeling_points = {"stressed": 1, "low": 2, "neutral": 3, "good": 4, "excellent": 5}
```

```
satisfaction_points = {"very unsatisfied": 1, "unsatisfied": 2, "neutral": 3,
```

```
    "satisfied": 4, "very satisfied": 5}
```

```
energy_points = {"low": 1, "medium": 2, "high": 3}
```

```
# these lists will hold one number per valid day
```

```
sleep_list = []
```

```
fitness_list = []
```

```
study_list = []
coding_list = []
class_list = []
free_time_list = []
total_tracked_list = []
feeling_list = []
satisfaction_list = []
energy_list = []
dates_list = []
```

```
def average(numbers):
    if len(numbers) == 0:
        return 0
    return sum(numbers) / len(numbers)
```

```
def correlation(list_a, list_b):
    # basic Pearson correlation, done by hand (no numpy)
    avg_a = average(list_a)
    avg_b = average(list_b)

    top = 0
    bottom_a = 0
    bottom_b = 0
    for i in range(len(list_a)):
        top = top + (list_a[i] - avg_a) * (list_b[i] - avg_b)
        bottom_a = bottom_a + (list_a[i] - avg_a) ** 2
```

```
bottom_b = bottom_b + (list_b[i] - avg_b) ** 2
```

```
bottom = (bottom_a * bottom_b) ** 0.5
```

```
if bottom == 0:
```

```
    return 0
```

```
return top / bottom
```

```
# ---- read the excel file, one row at a time ----
```

```
workbook = openpyxl.load_workbook(file_name, data_only=True)
```

```
sheet = workbook[sheet_name]
```

```
for row in sheet.iter_rows(min_row=start_row, values_only=True):
```

```
    log_date = row[0]
```

```
    if log_date is None:
```

```
        break # empty row, nothing more to read below this
```

```
    try:
```

```
        sleep = row[1]
```

```
        fitness = row[2]
```

```
        study = row[3]
```

```
        coding = row[4]
```

```
        class_min = row[5]
```

```
        other = row[7]
```

```
        free_time = row[9]
```

```
        feeling = str(row[10]).strip().lower()
```

```

satisfaction = str(row[11]).strip().lower()
energy = str(row[12]).strip().lower()

numbers_to_check = [sleep, fitness, study, coding, class_min, other, free_time]
for value in numbers_to_check:
    if not isinstance(value, (int, float)) or value < 0:
        raise ValueError("bad number in this row")

if feeling not in feeling_points:
    raise ValueError("feeling not recognised: " + feeling)
if satisfaction not in satisfaction_points:
    raise ValueError("satisfaction not recognised: " + satisfaction)
if energy not in energy_points:
    raise ValueError("energy not recognised: " + energy)

log_date = log_date.date()
if log_date < record_start:
    print("Skipping", log_date, "- looks like the sample row")
    continue

dates_list.append(log_date)
sleep_list.append(sleep)
fitness_list.append(fitness)
study_list.append(study)
coding_list.append(coding)
class_list.append(class_min)

```

```

free_time_list.append(free_time)

total_tracked_list.append(sleep + fitness + study + coding + class_min + other)

feeling_list.append(feeling_points[feeling])

satisfaction_list.append(satisfaction_points[satisfaction])

energy_list.append(energy_points[energy])


except (ValueError, TypeError) as error:

    print("Skipping a row because:", error)


# ---- calculate the indices ----

valid_days = len(dates_list)

study_and_class = [study_list[i] + class_list[i] for i in range(valid_days)]

feeling_total = [feeling_list[i] + satisfaction_list[i] + energy_list[i] for i in
    range(valid_days)]

TPI = average(coding_list)

AAI = average(study_and_class)

PhAI = average(fitness_list)

SRI = average(sleep_list)

ABI = average(free_time_list)

TUI = average(total_tracked_list)

EI = average(feeling_total) / 3

expected_days = (record_end - record_start).days + 1

days_in_window = 0

```

```

for d in dates_list:

    if record_start <= d <= record_end:

        days_in_window = days_in_window + 1

DCI = (days_in_window / expected_days) * 100


# PAI needs everything on the same 0-100 scale

TPI_s = (TPI / 1440) * 100
AAI_s = (AAI / 1440) * 100
PhAI_s = (PhAI / 1440) * 100
SRI_s = (SRI / 1440) * 100
TUI_s = (TUI / 1440) * 100
EI_s = (EI / 5) * 100


PAI = (0.15 * TPI_s + 0.20 * AAI_s + 0.15 * PhAI_s + 0.20 * SRI_s
      + 0.15 * TUI_s + 0.10 * EI_s + 0.05 * DCI)


# a few relationships between my habits

coding_vs_energy = correlation(coding_list, energy_list)
sleep_vs_energy = correlation(sleep_list, energy_list)
study_vs_satisfaction = correlation(study_list, satisfaction_list)


# ---- print my report ----

print("\n----- MY DATA, MY STORY -----")

print("Valid days used:", valid_days)


print("\nTPI (coding, min/day)   :", round(TPI, 1))

```

```
print("AAI (study+class, min/day) :", round(AAI, 1))
```

```
print("PhAI (fitness, min/day) :", round(PhAI, 1))
```

```
print("SRI (sleep, min/day) :", round(SRI, 1))
```

```
print("ABI (free time, min/day) :", round(ABI, 1))
```

```
print("TUI (total tracked, min) :", round(TUI, 1))
```

```
print("EI (experience, 1-5) :", round(EI, 2))
```

```
print("DCI (data continuity, %) :", round(DCI, 1))
```

```
print("PAI (overall score, /100) :", round(PAI, 1))
```

```
print("\nCoding vs Energy : r =", round(coding_vs_energy, 2))
```

```
print("Sleep vs Energy : r =", round(sleep_vs_energy, 2))
```

```
print("Study vs Satisfaction : r =", round(study_vs_satisfaction, 2))
```

```
print("\nMy findings:")
```

```
print("- I averaged", round(SRI), "minutes of sleep and", round(TPI), "minutes of  
coding per day.")
```

```
print("- My data continuity was", round(DCI, 1), "% over the recording window.")
```

```
if abs(sleep_vs_energy) > abs(coding_vs_energy) and abs(sleep_vs_energy) >  
abs(study_vs_satisfaction):
```

```
    print("- Sleep and energy showed the strongest link out of the three I checked.")
```

```
elif abs(coding_vs_energy) > abs(study_vs_satisfaction):
```

```
    print("- Coding and energy showed the strongest link out of the three I checked.")
```

```
else:
```

```
    print("- Study and satisfaction showed the strongest link out of the three I  
checked.")
```